



UFRN - Universidade Federal do Rio Grande do Norte

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 14 - PROGRAMAÇÃO EM MEMÓRIA DISTRIBUÍDA

1. Enunciado

A tarefa consiste em implementar uma troca de mensagens entre exatamente dois processos MPI: o processo de rank 0 envia uma mensagem ao processo de rank 1, que responde imediatamente com o mesmo conteúdo (ping-pong). O tempo de múltiplas trocas consecutivas deve ser medido com `MPI_Wtime`, para tamanhos de mensagem que vão de 8 bytes a pelo menos 1 MB. A partir dessas medições, o tempo deve ser representado graficamente em função do tamanho da mensagem, identificando as regiões em que a latência domina o custo de comunicação e aquelas em que o custo de transferir os bytes passa a ser o fator principal.

2. Fundamentação

No modelo de programação MPI, cada processo possui seu próprio espaço de endereçamento; toda troca de dados entre processos ocorre explicitamente por meio de chamadas de biblioteca, e não por memória compartilhada. As rotinas empregadas neste experimento são:

2.1. Inicialização e finalização

- `MPI_Init`: inicializa o ambiente MPI; deve ser chamada antes de qualquer outra rotina MPI.
- `MPI_Finalize`: encerra o ambiente MPI, liberando os recursos internos alocados pela biblioteca.

2.2. Identificação dos processos

- `MPI_Comm_rank`: retorna o identificador (rank) do processo dentro do comunicador `MPI_COMM_WORLD` — neste experimento, 0 ou 1.
- `MPI_Comm_size`: retorna o número de processos no comunicador — neste experimento, verificado como sendo exatamente 2 antes de prosseguir.

2.3. Comunicação ponto a ponto

- `MPI_Send`: envia uma mensagem no modo padrão (blocking send), endereçada a um rank de destino.
- `MPI_Recv`: recebe a mensagem correspondente a um envio, bloqueando até que os dados estejam disponíveis.

Na tarefa implementada, o rank 0 primeiro envia e depois recebe a resposta; o rank 1 primeiro recebe e depois responde com o mesmo buffer. Essa ordem de envio seguido de recebimento em um lado, recebimento seguido de envio no outro é o que evita deadlock entre os dois processos, já que a cada instante há sempre um envio pendente com um recebimento correspondente já postado do outro lado.

3. Experimento

Os testes foram executados no cluster NPAD (UFRN), partição amd-512, via agendador Slurm, em duas configurações distintas, para isolar o efeito do meio físico de comunicação sobre a latência e a largura de banda medidas:

- Configuração 1 nó: as duas tarefas MPI alocadas no mesmo nó físico (`--nodes=1, --ntasks=2`), comunicando-se via memória compartilhada;
- Configuração 2 nós: uma tarefa por nó (`--nodes=2, --ntasks=2`), forçando a comunicação a trafegar pela rede de interconexão do cluster.

Em ambos os casos o programa foi compilado no próprio job com `mpicc` e executado com `mpirun`. O código completo e os scripts Slurm utilizados estão reproduzidos nos Anexos A e B.

Para cada tamanho de mensagem — variando de 8 bytes a 8 MB, em potências de 2 — os dois processos alocam um único buffer (reaproveitado entre todos os tamanhos) e o preenchem antes de iniciar as medições. Uma barreira (`MPI_Barrier`) sincroniza os dois processos imediatamente antes do início da contagem de tempo. O processo 0 então mede, com `MPI_Wtime`, 1000 pares consecutivos de envio e recebimento (uma ida e uma volta completas por par). A alocação do buffer, seu preenchimento, a barreira, a impressão dos resultados e a finalização do MPI ficam fora do intervalo cronometrado.

Como cada par ida-e-volta transfere a mensagem duas vezes (uma em cada sentido), o tempo médio reportado na Tabela 1 e na Tabela 2 é obtido dividindo o tempo total pelo dobro do número de iterações (2×1000), e corresponde a uma estimativa do tempo de transferência em um único sentido — sob a hipótese de que o custo de enviar e o de responder são aproximadamente simétricos. Houve uma execução por tamanho em cada configuração; não foram medidas repetições independentes do experimento completo, apenas as 1000 trocas internas usadas para compor cada média.

4. Resultados

4.1. Configuração 1 nó (memória compartilhada)

Tamanho (bytes)	Tempo médio/msg (s)	Banda efetiva (MB/s)
8	4.178638e-06	1.8258
16	1.241695e-07	122.8868
32	1.194855e-07	255.4082
64	1.173665e-07	520.0390
128	4.971690e-07	245.5308
256	2.264880e-07	1077.9407
512	2.802490e-07	1742.3122
1024	3.611620e-07	2703.9459
2048	5.314385e-07	3675.1666
4096	8.576600e-07	4554.5438
8192	7.568950e-07	10321.7751
16384	1.347696e-06	11593.8652
32768	2.104475e-06	14849.3092
65536	3.885153e-06	16086.8805
131072	7.625048e-06	16393.3394
262144	9.326832e-06	26804.3855
524288	3.048141e-05	16403.4407
1048576	5.445717e-05	14217.5805
2097152	2.019266e-04	9904.5874
4194304	3.833163e-04	10435.2468
8388608	7.565226e-04	10574.7003

Tabela 1 — Tempo médio por mensagem e largura de banda efetiva, 1 nó.

Tarefa 14 — 1 nó (memória compartilhada)

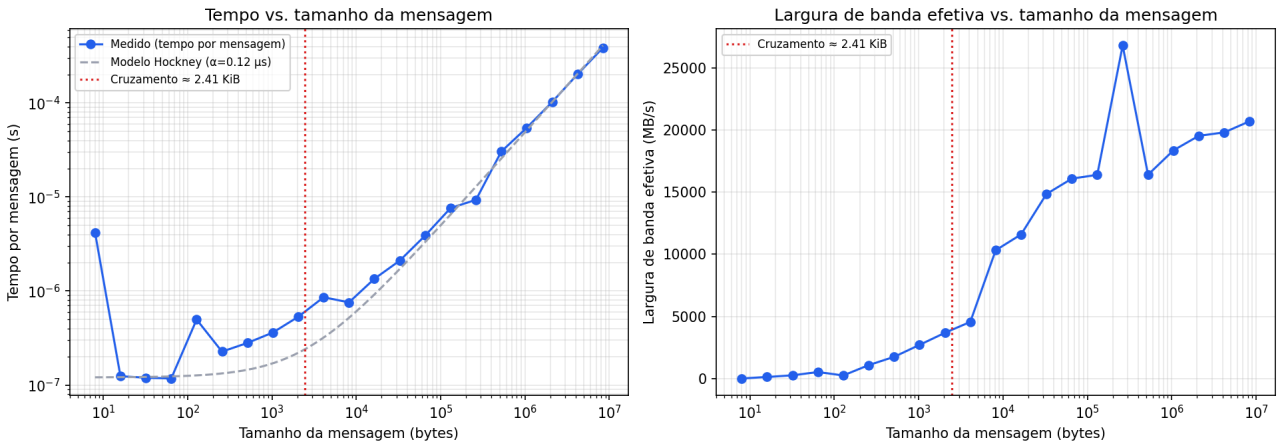


Figura 1 — Tempo por mensagem e largura de banda efetiva vs. tamanho, 1 nó, com ajuste ao modelo de Hockney.

4.2. Configuração 2 nós (rede)

Tamanho (bytes)	Tempo médio/msg (s)	Banda efetiva (MB/s)
8	1.635494e-06	4.6649
16	1.636741e-06	9.3227
32	1.814415e-06	16.8195
64	1.956538e-06	31.1955
128	1.977391e-06	61.7330
256	3.152546e-06	77.4424
512	2.633074e-06	185.4415
1024	3.037870e-06	321.4629
2048	3.579604e-06	545.6261
4096	4.784237e-06	816.4833
8192	6.555918e-06	1191.6715
16384	9.106862e-06	1715.7391
32768	1.410406e-05	2215.6733
65536	2.251223e-05	2776.2688
131072	3.841687e-05	3253.7790
262144	5.762678e-05	4338.2611
524288	5.742137e-05	8707.5589
1048576	9.472331e-05	10557.0638

Tamanho (bytes)	Tempo médio/msg (s)	Banda efetiva (MB/s)
2097152	1.821735e-04	10978.5431
4194304	3.532769e-04	11322.5624
8388608	6.980299e-04	11460.8264

Tabela 2 — Tempo médio por mensagem e largura de banda efetiva, 2 nós.

Tarefa 14 — 2 nós (rede)

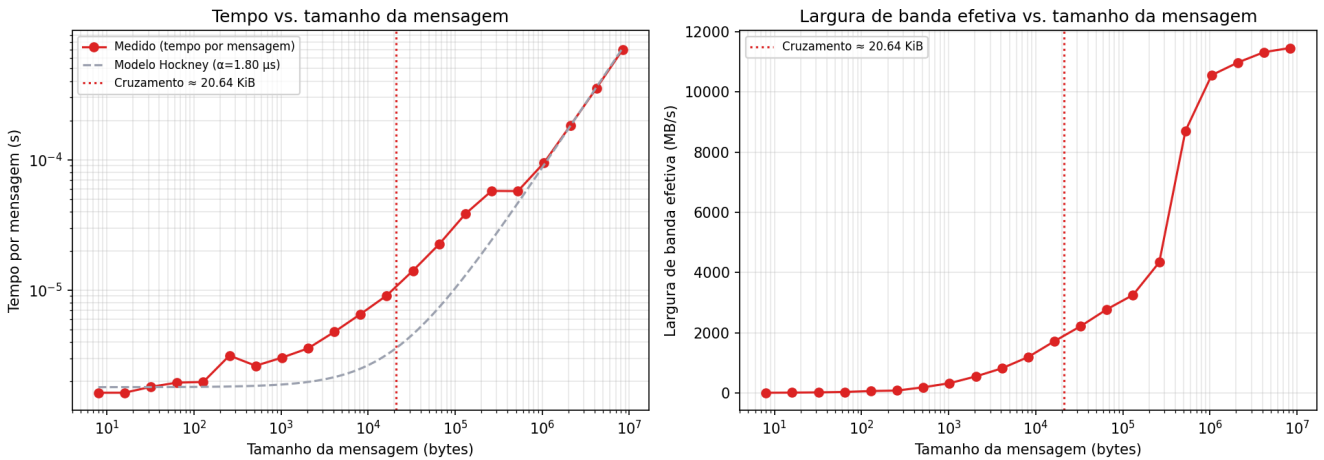


Figura 2 — Tempo por mensagem e largura de banda efetiva vs. tamanho, 2 nós, com ajuste ao modelo de Hockney.

4.3. Comparação entre as duas configurações

Tarefa 14 — Comparação intra-nó vs. inter-nó (NPAD, amd-512)

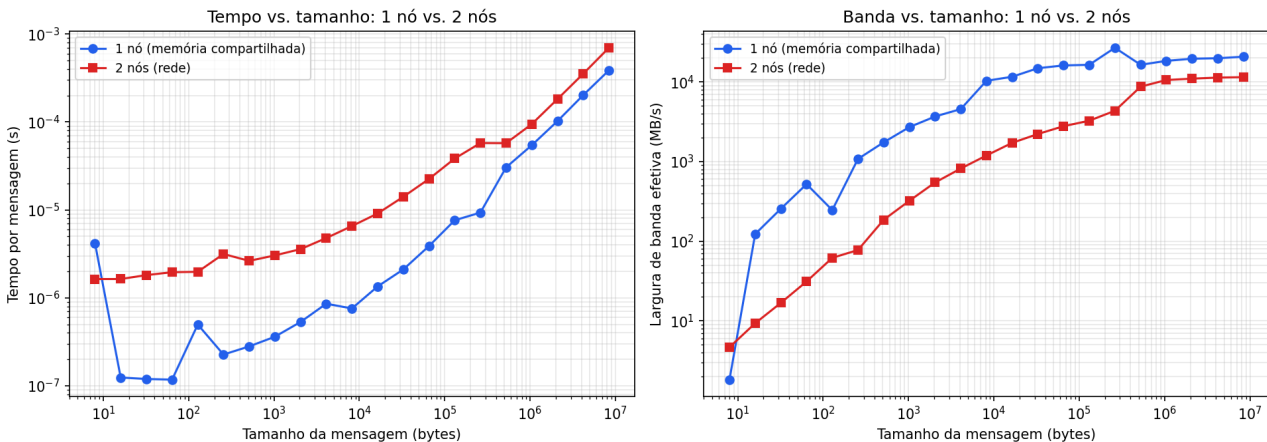


Figura 3 — Comparação direta entre as configurações de 1 nó e 2 nós.

Entre 8 e 64 bytes, em ambas as configurações, o tempo médio por mensagem varia pouco: é a região em que a latência fixa domina o custo de comunicação. A partir de algumas dezenas de KB, o tempo passa a crescer de forma aproximadamente linear com o tamanho, indicando que a transferência dos

bytes passa a dominar. Como os dois ranks da configuração 1 nó foram alocados no mesmo nó, aqueles dados isoladamente não demonstrariam a largura de banda de uma rede entre nós — daí a importância da segunda configuração, com um processo por nó, para caracterizar também o regime de rede real do cluster.

5. Análise dos Resultados

5.1. Modelo de Hockney

Para localizar com mais precisão a transição entre os dois regimes, o tempo médio por mensagem foi ajustado ao modelo de Hockney:

$$T(n) = \alpha + n / \beta$$

onde α é a latência fixa (tempo de uma mensagem de tamanho tendendo a zero) e β é a largura de banda assintótica do canal. O tamanho de cruzamento $n^* = \alpha \cdot \beta$ marca o ponto em que os dois termos se igualam: abaixo dele, o tempo de comunicação é dominado pela latência; acima dele, pela largura de banda.

Configuração	Latência (α)	Banda assintótica (β)	Cruzamento (n^*)
1 nó (memória compartilhada)	$\approx 0,12 \mu\text{s}$	$\approx 20,5 \text{ GB/s}$	$\approx 2,41 \text{ KiB}$
2 nós (rede)	$\approx 1,80 \mu\text{s}$	$\approx 11,7 \text{ GB/s}$	$\approx 20,64 \text{ KiB}$

Tabela 3 — Parâmetros do modelo de Hockney ajustados para cada configuração.

5.2. Efeito do meio de comunicação

A latência medida entre 2 nós é $\approx 1,80 \mu\text{s}$ já em um único nó $\approx 0,12 \mu\text{s}$, refletindo o custo adicional de atravessar a pilha de rede — driver, protocolo de transporte e adaptador físico — em vez de uma simples cópia em memória. A largura de banda assintótica também é menor entre 2 nós $\approx 11,7 \text{ GB/s}$ contra $\approx 20,5 \text{ GB/s}$ em apenas um nó, consistente com o teto de banda mais restritivo da interconexão física do cluster frente ao barramento de memória interno de um nó.

Como consequência, o ponto de cruzamento n^* se desloca de $\approx 2,41 \text{ KiB}$ (1 nó) para $\approx 20,64 \text{ KiB}$ (2 nós): quanto maior a latência do meio, maior precisa ser a mensagem para que o custo de transferi-la justifique o overhead fixo de cada troca. Na prática, isso implica que aplicações MPI que trocam muitas mensagens pequenas entre nós sofrem proporcionalmente mais com a latência de rede do que equivalentes executados dentro de um único nó.

5.3. Ruído experimental

Na série de 1 nó, o tempo medido para 8 bytes ($4,18 \times 10^{-6} \text{ s}$) destoa do restante da tendência, ficando bem acima do valor esperado pelo modelo ($\approx 0,12 \mu\text{s}$). Como não houve iterações de aquecimento antes da primeira medição, esse desvio é atribuído a custos de inicialização interna do MPI e de paginação de

memória que só ocorrem na primeira chamada de MPI_Send/MPI_Recv de todo o programa. Os pontos de 128 bytes e 524 288 bytes também se desviam ligeiramente da curva suave, consistentes com ruído do sistema operacional (jitter do escalonador) no nó compartilhado do cluster — efeito que a média sobre 1000 repetições já amortece parcialmente, mas não elimina por completo, já que houve apenas uma execução do experimento por tamanho.

6. Conclusão

O experimento confirmou o comportamento previsto pelo modelo de Hockney para comunicação ponto a ponto em MPI: um custo fixo de latência somado a um custo proporcional ao tamanho da mensagem dividido pela largura de banda do canal. Na configuração de 1 nó, a latência estimada foi de aproximadamente 0,12 μ s e a largura de banda assintótica de aproximadamente 20,5 GB/s, valores característicos de comunicação via memória compartilhada. Na configuração de 2 nós, a latência subiu para aproximadamente 1,80 μ s e a largura de banda caiu para aproximadamente 11,7 GB/s, refletindo o custo de atravessar a rede de interconexão física do cluster.

O ponto de cruzamento entre os dois regimes foi estimado em aproximadamente 2,4 KB para comunicação em um nó e aproximadamente 20,6 KiB para comunicação entre os dois nós, evidenciando que o tamanho de mensagem a partir do qual a largura de banda passa a dominar depende diretamente da latência do meio de comunicação utilizado. Os dois pontos de ruído identificados na série de 1 nó reforçam a importância de incluir iterações de aquecimento e, idealmente, repetições independentes do experimento em versões futuras deste benchmark, para reduzir a variância nas medições de tamanhos pequenos de mensagem.

Anexo A — Código completo (main.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define ITERACOES 1000

int main(int argc, char** argv) {
    int rank, size;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Garante que o programa é executado com exatamente 2 processos
    if (size != 2) {
        if (rank == 0) {
            printf("Erro: A Tarefa 14 exige exatamente 2 processos!\n");
        }
        MPI_Finalize();
        return 0;
    }

    // Tamanho máximo de 8 MB
    long max_bytes = 8 * 1024 * 1024;
    char* buffer = (char*) malloc(max_bytes);

    if (rank == 0) {
        printf("| Tamanho_Bytes    | Tempo_Medio_Msg_s    |    Largura_Banda_MBs    | \n");
    }

    // Variando o tamanho da mensagem (de 8 bytes até 8 MB)
    for (long bytes = 8; bytes <= max_bytes; bytes *= 2) {
        // Sincroniza os dois processos antes de iniciar a medição do cronômetro
        MPI_Barrier(MPI_COMM_WORLD);
        double start = MPI_Wtime(); // Iniciar a medição

        for (int i = 0; i < ITERACOES; i++) {
            if (rank == 0) {
                MPI_Send(buffer, bytes, MPI_BYTE, 1, 0, MPI_COMM_WORLD);
                MPI_Recv(buffer, bytes, MPI_BYTE, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            } else if (rank == 1) {
                MPI_Recv(buffer, bytes, MPI_BYTE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                MPI_Send(buffer, bytes, MPI_BYTE, 0, 0, MPI_COMM_WORLD);
            }
        }

        double finish = MPI_Wtime(); // Finalizar a medição

        if (rank == 0) {
            double tempo_total = finish - start;
            // Cada iteração do laço realiza 2 mensagens (uma ida e uma volta)
            double tempo_uma_mensagem = tempo_total / (2.0 * ITERACOES);
            double bytes_para_MB = bytes / (1024.0 * 1024.0);
            double largura_banda = bytes_para_MB / tempo_uma_mensagem;

            printf("|    %ld    |    %e    |    %f    | \n", bytes, tempo_uma_mensagem, largura_banda);
        }
    }

    free(buffer);
    MPI_Finalize();
}
```

```
    return 0;  
}
```

Anexo B — Scripts Slurm

B.1. Configuração 1 nó

```
#!/bin/bash
#SBATCH --job-name=tarefa_14
#SBATCH --time=0-0:10
#SBATCH --partition=amd-512
#SBATCH --nodes=1
#SBATCH --ntasks=2
#SBATCH --cpus-per-task=1
#SBATCH --output=%x_%j.out
#SBATCH --error=%x_%j.err

mpicc -o main main.c
mpirun main
```

B.2. Configuração 2 nós

```
#!/bin/bash
#SBATCH --job-name=tarefa_14
#SBATCH --time=0-0:10
#SBATCH --partition=amd-512
#SBATCH --nodes=2
#SBATCH --ntasks=2
#SBATCH --cpus-per-task=1
#SBATCH --output=%x_%j.out
#SBATCH --error=%x_%j.err

mpicc -o main main.c
mpirun main
```